

MTSC2025

中国互联网测试开发大会

TESTING SUMMIT CONFERENCE CHINA 2025

上海
站

质效革新·智领未来

2025/7/12 | 上海喜来登由由大酒店 主办方: TesterHome



游戏性能压测 AI智能化实践分享

- 游族网络



主讲人: 乌贼王(许学松)



时间: 2025.07

目录

CONTENT

01

游戏压测简介和价值

02

传统压测痛点分析

03

压测平台AI化演进路径

04

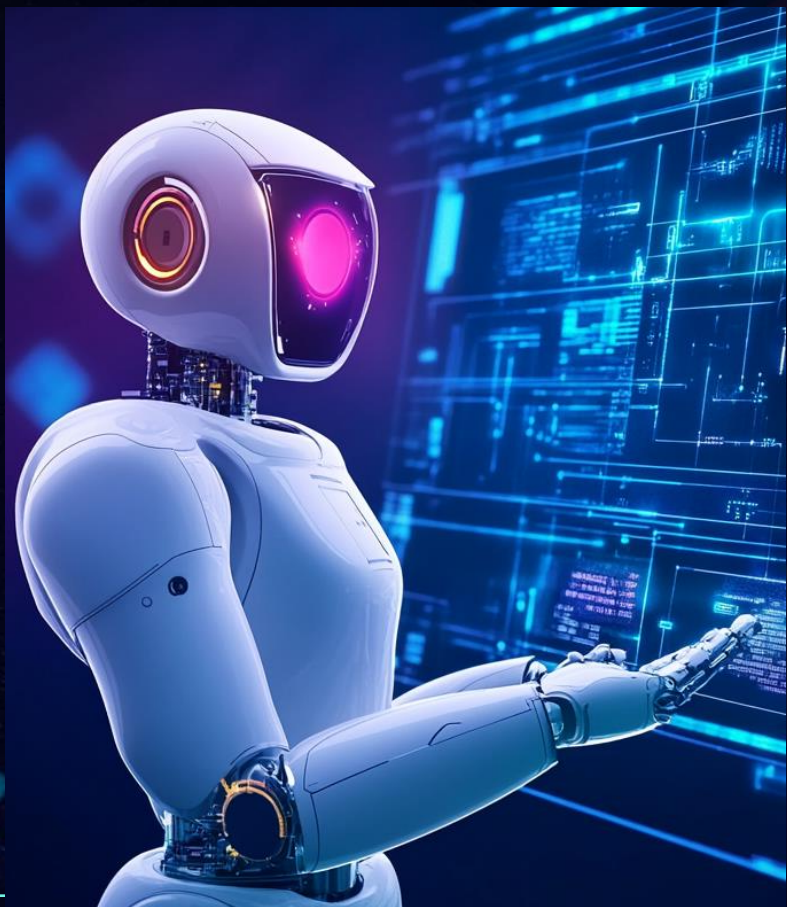
AI在游戏压测中的方案和成效

05

未来规划与挑战

PART 01

游戏压测简介和价值





游戏压测核心价值

资源瓶颈定位

定位瓶颈、减少资源浪费，确保CPU $\leq 70\%$ 、内存泄漏率=0%，提升系统性能与资源利用效率。

稳定性保障

预防宕机、提升容错能力，实现错误率 $< 1\%$ 、断线重连成功率 $\geq 99.9\%$ ，保障系统稳定运行。

玩法验证

确保新功能上线流畅性，保障帧率 $\geq 30\text{FPS}$ 、响应时间 ≤ 1 秒，提升玩家体验。

经济效益

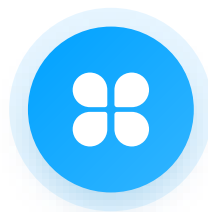
降低硬件与运维成本，实现服务器采购成本降低30%-70%，提升经济效益。



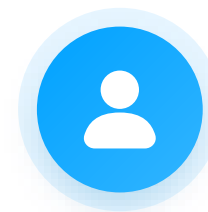
资源瓶颈定位



CPU/内存泄漏检测



数据库性能优化



硬件资源规划



稳定性保障



异常自动处理

在压测过程中模拟多种异常场景，验证重试策略与熔断机制的有效性，确保系统在面对突发故障时能够自动恢复。



多进程负载均衡

利用压测数据优化进程负载均衡算法，确保各进程负载均衡，避免因负载不均导致的性能瓶颈。



长期稳定性验证

开展8小时持续压测，模拟玩家在线峰值（如赛季活动开服），确保内存无泄漏（如RSS值稳定）、DELAY值<1秒，验证系统的长期稳定性。



多人与高频场景的性能验证



高频操作验证

通过压测确保服务器帧率 $\geq 30\text{FPS}$ 、
响应时间 ≤ 0.5 秒，保障玩家操作的流畅性。



数据库压力模拟

利用压测数据优化数据库事务处理机制，提升数据库性能，保障大型玩法的顺利进行。



客户端性能适配

通过压测排查客户端内存泄漏（多人场景如未释放的纹理资源），减少闪退率，提升玩家的游戏体验。



精细化成本控制

第三方服务选型

基于压测数据评估不同第三方服务的性能与成本，为服务选型提供科学依据。

01

流量预估模型

利用压测数据优化流量预估模型，提升模型准确性，实现精细化成本控制。

02

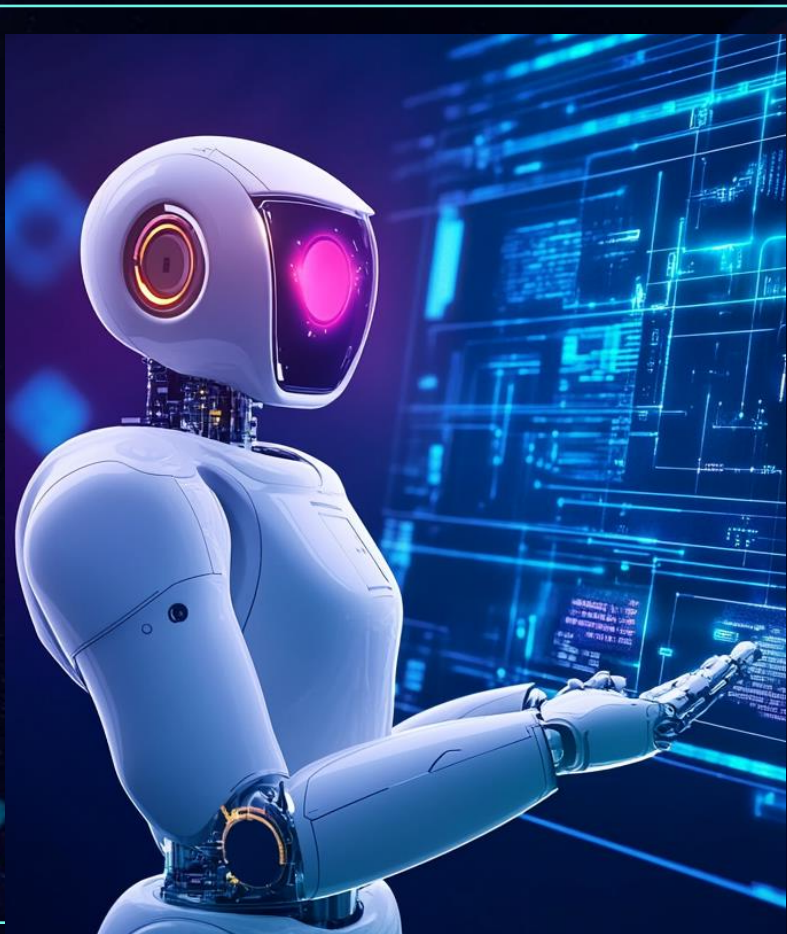
自动化运维体系

集成ELK日志分析系统，实时标记异常热点（如技能释放服务超时），并推送风险信息，助力快速定位与解决问题。

03

PART 02

传统压测痛点分析



人力成本高：开发周期与敏捷迭代矛盾

01

人肉沟通，编码耗时

压测脚本需逐行模拟协议时序逻辑，测试周期2个月起步，功能玩法沟通成本占测试总投入60%及以上。

不同项目编解码方式多样（C，Java，GO，C#，NodeJs），且大部分都是自定义协议规则，导致抓包工具无法适配和统一化处理。

规则不一，代码多样化

02

03

协议频更，周期压缩

协议的任何变动都需重构代码，测试团队长期面临“开发-测试-维护”循环依赖，导致压测结果失真，且压测一般都是集中在项目上线，测试周期压缩严重。

工具局限性：动态协议与多架构适配难题

现有工具无法处理动态密钥交换、加解密等流程，无法实时查看协议内容。

现有工具缺少解析与适配能力，难以应对复杂协议与多项目需求

客户端工具操作不便，数据无法共享，较难协作，改造难成本高

压测门槛高：技能要求与数据洞察瓶颈



各语言特性
常用中间件
常用数据集
游戏服架构



游戏玩法
性能瓶颈
测试场景
问题排查
独立部署



性能指标
内存泄漏
性能衰减
CPU波动
负载不均衡
稳定性设计
全链路规划

PART 03

压测平台AI化演进路径





AI前压测流程

01.



02.



03.

开发测沟通

功能玩法涉及到多协议交互，需要和开发沟通设计的规则，原理和涉及的协议列表

脚本编写调试

构建新项目环境，纯手工编写测试用例脚本，复杂场景需要多次调试和反复沟通

测试压测执行

压测执行出现问题后需要多次配合开发进行复测，调试，压力调整



流程缺点和不足

**重度
依赖后端**

**工作
重复性高**



AI架构技术演进

01.

协议捕获基建

目标：构建协议捕获基础能力。

关键技术成果：gnet 框架改造（收发包延迟 < 50ms）

，实时解析协议并保存

02.

AI 代码生成器

目标：实现 AI 辅助代码生成

关键技术成果：DeepSeek 模型集成（响应时间 < 60s）

03.

自动化压测平台

目标：打造自动化压测平台。

关键技术成果：游测平台根据玩法自动生成压测代码



AI架构优势

**协议获取
自主可控**

**极大降低
依赖度**

**压测代码
可部分生成**



PART 04

AI在游戏压测中的 方案和成效



业务架构方案

接入层

业务系统平台

创建压测任务

GitLab仓库关联

监控配置

发压机和压力配置

应用层

AI代码生成

Message模板

Logic模板

Main模板

代码生成规则

具体业务需求配置

代码评审机制

自动审核

人工审核

知识库配置

压测仓库

通用案例

性能指标标准

领域层

AI服务

DeepSeek

火山云

GPT4

通义千问

数据清洗

清洗规则

去重逻辑

游戏关联数据

基建层

底层服务

代理服务

监控服务

基建压测基础服务

数据存储

MongoDB

DB

DB



核心模块概述

协议采集层

代理服务器集群：基于gnet框架实现高并发转发，支持TCP/UDP/WS协议，满足大规模并发需求。

动态密钥管理：维护map[clientID][sessionKey]，支持中间人解密（如RSA握手后替换AES密钥），确保协议数据完整捕获。

数据处理层

协议存储：MongoDB集群存储原始协议（时间戳、方向、原始报文），支持海量数据存储与高效查询。

AI代码生成

数据清洗：过滤心跳包、重复请求，提取关键字段（如enemy_id），提升数据质量，为AI生成提供可靠输入。

模型服务：输入清洗后协议+玩法描述，输出协议对、逻辑链、代码变量，实现智能解析与代码生成。

模板样例：渲染message/logic/main代码，自动提交GitLab分支，提升代码生成效率与规范性。

任务执行层

GitLab：实现代码的自动化提交

压测调度：生成压测代码后，基于压测平台，自动执行压测任务



协议追踪模块



代理技术

多路复用：单代理支持多客户端连接，协程池隔离不同玩家链路，确保数据互不干扰。

协议解析：复用压测脚本解包模块，按协议号映射结构体，适配多种协议格式。

数据保存：协议解析后，异步保存协议数据



安全处理

中间人 (MITM)：拦截登录协议，替换加密参数，维护两套秘钥池，确保协议数据安全捕获。

流量镜像：实时复制双端流量，不影响正常业务，实现无感知协议追踪。



AI代码生成模块

根据抓取到的协议，借助AI生成的压测逻辑

竞技场战斗

```
1  {
2    "protoItems": [
3      ["ArenaInfoRequest", "ArenaInfoResponse"],
4      ["RefreshEnemyListRequest", "RefreshEnemyListResponse"],
5      ["ArenaChallengeRequest", "ArenaChallengeResponse"]
6    ],
7    "process": "玩法名: 竞技场战斗玩法、玩法流程: 初始化逻辑: 1、基础数据加载 (推送协议: HeroesNotify, SCUpdateItem, SCUpdateResource, SCUpdateDailyTaskProcess) 主玩法逻辑: 1、获取竞技场信息 (请求: ArenaInfoRequest->响应: ArenaInfoResponse) 2、刷新敌人列表 (请求: RefreshEnemyListRequest->响应: RefreshEnemyListResponse) 3、发起挑战 (请求: ArenaChallengeRequest参数enemy_id/num来自刷新列表响应->响应: ArenaChallengeResponse含战斗结果和排名变化) "
8  }
```



AI代码生成模块

Msg代码（完全准确）：

```
package equipEnhanceMsg

import "github.com/vinson/tester"

func init() {
    var message Message
    tao.Register(message.ResponseCommand(), deserializeMessage, processMessage)
}
```

Logic代码（完全准确）：

```
package logic

import "github.com/vinson/tester"

type equipEnhanceLogic struct {
    basicLogic
    userLogic
    ctx boomer.Context
}

func NewEquipEnhanceLogic() equipEnhanceLogic {
    return &equipEnhanceLogic{}
}

func (l *equipEnhanceLogic) Enhance(ctx boomer.Context) {
    conn := l.GetClientConn(ctx)
    req := &cs.E25_Equip_Enhance{
        id: 1,
        CostEquips: []uint64{2, 3},
    }
    equipEnhanceMsg.RequestMessage(conn, ctx, req)
}
```

Main代码（完全准确）：

```
package main

import "github.com/vinson/tester"

func main() {
    flag.Parse()

    userLogic := logic.NewUserLogic()
    loginLogic := logic.NewLoginLogic()
    equipLogic := logic.NewEquipEnhanceLogic()

    taskInit := &boomer.Task{
        Name: "taskInit",
        Weight: 1,
        Fn: func(ctx boomer.Context) {
```

Message层：自动生成Protobuf结构体定义，从协议号映射，适配不同项目需求。

Logic层：通过示例代码展示竞技场战斗的逻辑流程，实现协议交互与参数传递。

Main层：控制并发模型，适配大规模压测场景。

模板



平台演示—1、代理服务器配置

配置代理游戏服信息

代理服配置管理

代理服务器配置维护

新增代理服

ID	代理服名称	文件版本	状态	操作
107	代理服	3	已停止	编辑 开启 删除

编辑代理服

* 代理服名称

s 代理服

* 代理服端口

3

* 游戏服地址

124.

* 游戏服ID

* Proto文件版本

proto 0211155027

取消

确定



平台演示—2、配置压测任务

平台上配置AI压测任务

①

基础信息

②

发压配置

③

其他配置

基础信息

有意义的任务名称和描述有助于后期快速检索任务和报告的查看，所以在填任务名称时候请尽量使用有测试场景和测试目的的描述。

* 任务名称

slime-主线任务

* 仓库地址

https://github.com/xxl/jmeter-plugins-tester

代码方式

☐ 已有代码

☒ AI生成

* 代码参考

feat-vinson

EnterGameMessage

EnterGame.go

SoloBattle

协议数据

slime代理服

vinl

2025-07-02 17:20:00

2025-07-02 17:28:00

专项申请单

质量管理部-测试项目-平台接口压测定时任务-20230410-复制

任务描述

主线任务

任务描述: 辅助AI理解需要生成的代码的业务逻辑



执行AI代码生成逻辑 代码自动提交到仓库

压测是基于Jmeter框架，通过设置相关压力值和接口参数来模拟用户的大压力行为下的检测接口稳定性。在压测同类接口时，通过压测工具可以模拟出多用户同时访问接口，从而检测出接口在高并发下的性能表现。

协议数据编辑

```

[["pb","B
c: server.BattleResultDataPb), de
sion: g
fgVers
ion: (su
b","BattleCheckRt
id: it64
resu
cli
's
server.P
...
h":
...
ime{ R
(int64))'
b","
cns
er
...
T
int64
e: (u
neN
e: (s
ng)
meZone
V
ic
32))}),"pb","
tag
...
ageBattle
F
Stag.
lient2s
er.StageF
"),
o","K
...
r{ B
a
...
o","
Sta
Mon
erRt{ Roo
mClear.
(int32))"}]]

```

AI逻辑编辑

玩法名：主线任务战斗玩法、玩法流程：初始化逻辑：1、时间同步
(请求：FeTime -> 响应：FeTimeRt) 主玩法
逻辑：1、开启战斗 (请求：StartBattle -> 响应：StartBattleRt, 推送协议：StartBattleNotice) 2、击杀怪物
(请求：KillMonster -> 响应：KillMonsterRt) 3、提交战斗结果校验 (请求：BattleCheck -> 响应：BattleCheckRt, 推送协议：StageBattleNotice)

取消

确认

质效革新·智领未来



输出核心数据表

核心监控信息

输出核心数据表

核心监控信息



AI效率提升

单用例开发时效

改造前：开发耗时2小时

改造后：仅需0.5小时

协议覆盖率

改造前： 50%

改造后： 80%

AI代码准确率

最低达： 30%



已验证场景

某游戏 - 武擂玩法压测

代码生成准确率: 60%

业务逻辑覆盖度: 40%

某游戏 - 好友系统压测

代码生成准确率: 50%

业务逻辑覆盖度: 60%

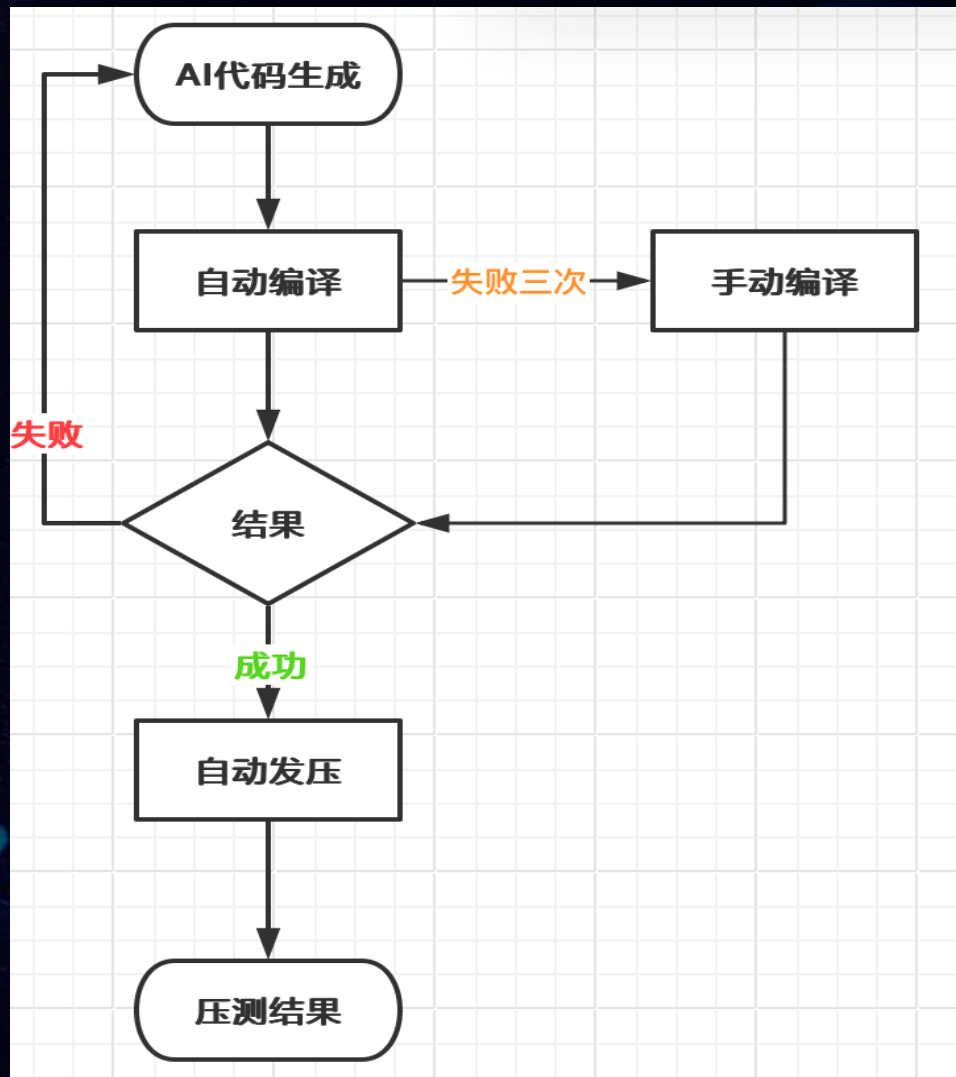
人工修正耗时 < 10分钟



PART 05

未来规划与挑战

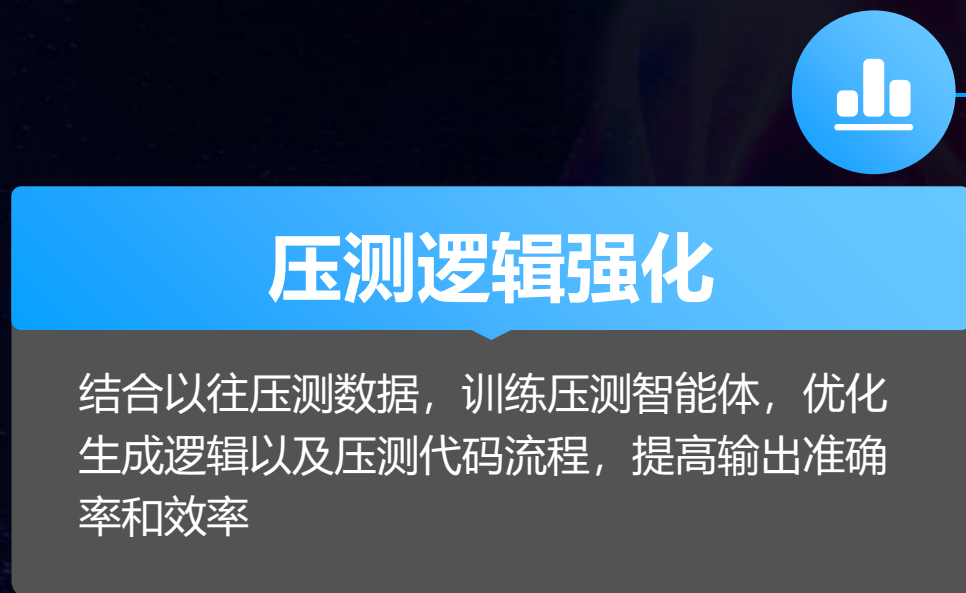
未来规划



半自动压测

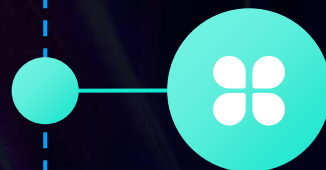
代码生成后自动编译，编译失败后则重新生成压测代码，减少人工干预，如果失败超过3次，则通知人工审核处理。编译成功后，自动发压脚本验证。

未来规划





未来规划



压测报告AI解读

基于生成的报告AI化解读，梳理出重点问题，并给出相应的风险以及对应的解决方案参考



待解决挑战

复杂协议关联推理

玩法匹配

跨服场景

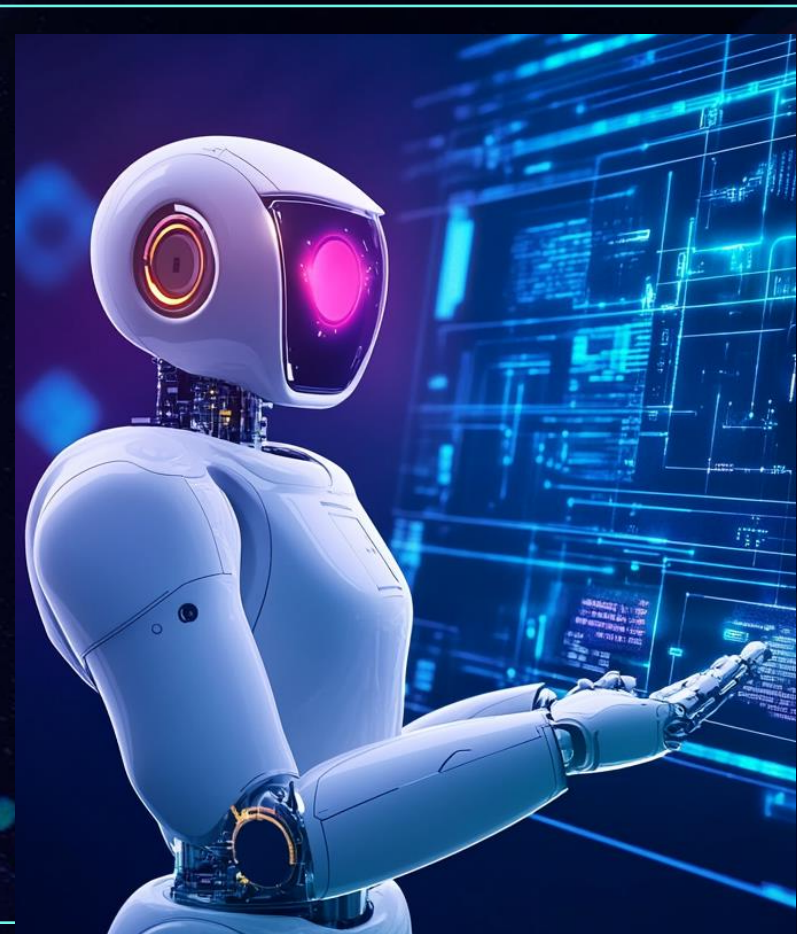
复杂协议关联

长事务逻辑生成

20 + 协议连续调用场景

TOP 90 协议混压测试

N小时的稳定性场景构建



谢谢大家